Linux Troubleshooting — Scenario #7: "The Server Can't Reach Anything by Name"

DevOps on-call runbook • Syed Aftab • Ubuntu (WSL) • DNS + resolv.conf + dig + journalctl • Reference for real incidents & interviews

The alert (7:05 AM): since last night's maintenance, the app can't reach its payment API and `apt update` fails with `Temporary failure in name resolution`. Other servers are fine.
Root cause found: during maintenance, `/etc/resolv.conf` — a **managed link** to WSL's generated file — was deleted and replaced with a hand-written file pointing to `nameserver 192.0.2.53`, which never answers. **Fix:** restore the link; keep the bad file as evidence.

Network problems are layered. The skill is testing **bottom-up** and stopping at the first layer that fails — here, everything worked **by IP**, only **names** failed, which points straight at DNS.

The methodology

#	Phase	Commands	Goal
1	Confirm	`sudo apt update`	Reproduce the error; notice it's slow to fail (a timeout)
2	Locate	`ip -br a`, `ip route`, `ping 8.8.8.8`, `ping google.com`	Which layer fails first? Interface, route, internet by IP, internet by name
3	Identify	`dig`, `ls -l / cat resolv.conf`, `dig @server`, `journalctl | grep`	Which DNS server, why it fails, does the good one work, who changed it
4	Fix	`mv` bad file aside, `ln -s` managed link back	Restore who manages the file, not just the IP
5	Verify	`ls -l`, `dig`, `ping google.com`	Link back, NOERROR from the right server, names resolve
6	Prevent	configure DNS in the right place, test after changes	Changes go through the manager; never hand-edit a managed file

BASELINE How DNS worked before the change

```
ls -l /etc/resolv.conf
lrwxrwxrwx 1 root root 20 Nov  3 06:56 /etc/resolv.conf -> /mnt/wsl/resolv.conf
cat /etc/resolv.conf
# This file was automatically generated by WSL. To stop automatic generation ... /etc/wsl.conf:
# [network]
# generateResolvConf = false
nameserver 10.255.255.254
search srivernj.org
dig google.com +short
142.250.188.14
```

Part	Meaning
`/etc/resolv.conf`	tells Linux **which DNS server to ask** to turn names into IPs
`l` + `-> /mnt/wsl/resolv.conf`	a **symbolic link** to a file WSL regenerates at every start — **managed**, don't hand-edit
`nameserver 10.255.255.254`	WSL's DNS proxy (lives on the loopback interface `lo`); it forwards questions to Windows
`search srivernj.org`	**search domain**: a short name like `fileserver` is also tried as `fileserver.srivernj.org`

Part	Meaning
`dig NAME +short`	ask DNS for a name's IP; `+short` = just the answer (install: `dnsutils`)

Real servers too: /etc/resolv.conf is usually managed by something else — `systemd-resolved` (link to `/run/systemd/resolve/stub-resolv.conf`, nameserver `127.0.0.53`), NetworkManager, or cloud-init. **Who owns the file** is part of every DNS fix.

The maintenance change (lab setup)

```
sudo rm /etc/resolv.conf
printf '# new DNS server - maintenance 11/02\nnameserver 192.0.2.53\n' \
    | sudo tee /etc/resolv.conf > /dev/null
```

Part	Meaning
`rm /etc/resolv.conf`	deleted the **link** only — WSL's real file /mnt/wsl/resolv.conf stayed intact
`printf '...\n...\n'`	print text; `\n` = new line → two lines
`| sudo tee FILE > /dev/null`	write it as root (the sudo-tee trick), hide the screen copy
`192.0.2.53`	the 192.0.2.0/24 range is **reserved for documentation** — nothing ever answers. Plays a mistyped or decommissioned DNS server

	Before	After
file type	link → /mnt/wsl/resolv.conf	a normal hand-written file
managed by	WSL (rewritten automatically)	nobody
nameserver	`10.255.255.254` answers	`192.0.2.53` never answers
search domain	`srivernj.org`	gone

Safety net in WSL: `wsl --shutdown` in Windows PowerShell, then reopen — WSL rebuilds the original link at startup.

STEP 1 Confirm — reproduce the error

```
sudo apt update
Ign:1 http://archive.ubuntu.com/ubuntu ... InRelease
Err:1 http://archive.ubuntu.com/ubuntu ... InRelease
  Temporary failure resolving 'archive.ubuntu.com'
```

`Temporary failure resolving` = "I couldn't turn this name into an IP". **Slow to fail** = the system is waiting on a DNS server that never replies — a timeout, itself a clue.

STEP 2 Locate — test the layers bottom-up

```
ip -br a
lo        UNKNOWN  127.0.0.1/8 10.255.255.254/32 ::1/128
eth0      UP       192.168.250.206/20 fe80::215:5dff:fe53:17a0/64
docker0   DOWN     172.17.0.1/16
ping -c 3 8.8.8.8
3 packets transmitted, 3 received, 0% packet loss ... avg 10.0 ms
ping -c 3 google.com
^C          <-- hung, never printed "PING google.com (IP)"
```

Layer	Command	Question	Result
1 Interface	`ip -br a`	card UP with an IP? (`-br` = brief, one line each)	eth0 UP 192.168.250.206 ✓
2 Routing	`ip route`	a `default via ...` gateway line?	✓ (proven by layer 3)
3 Internet by IP	`ping -c 3 8.8.8.8`	reach the internet by IP? (`-c 3` = 3 pings)	0% loss ✓
4 Internet by name	`ping -c 3 google.com`	same, but needs DNS first	**hung ✗**

Result pattern	Meaning
1 fails	no IP: network card / DHCP
1 OK, no default route	no way out: routing
1–2 OK, 3 fails	firewall / upstream network
1–3 OK, 4 fails	**network fine, only names fail** → **DNS** (our case)

Details in ip -br a: `10.255.255.254/32` on `lo` = the good DNS proxy lives on this machine's loopback. `docker0 DOWN` = Docker's bridge with the service stopped — **unrelated, a red herring**. ping google.com never printed its IP: it was stuck on the lookup **before sending a single ping**.

STEP 3 Identify — which server, does the good one work, who changed it

3a. Ask DNS directly

```
dig google.com
;; communications error to 192.0.2.53#53: timed out
;; communications error to 192.0.2.53#53: timed out
;; communications error to 192.0.2.53#53: timed out
; <<>> DiG 9.20.24-1ubuntu0.4-Ubuntu <<>> google.com
;; no servers could be reached
```

Clue	Meaning
`192.0.2.53`	**the DNS server it asked** — where did this come from?
`#53`	port 53, the DNS port
`timed out` ×3	asked 3 times, **silence** — not "name doesn't exist"
`no servers could be reached`	no DNS server answered → every lookup on the machine fails

dig says	Meaning
`timed out` / `no servers could be reached`	the **DNS server** is dead or unreachable (today)
`status: NXDOMAIN`	server answered: the **name** doesn't exist (typo, missing record)
`status: SERVFAIL`	server answered but couldn't resolve it (upstream problem)
`status: NOERROR` + ANSWER SECTION	working

3b. Where does 192.0.2.53 come from — and does the good server still work?

```
ls -l /etc/resolv.conf
-rw-r--r-- 1 root root 59 Nov  3 07:14 /etc/resolv.conf
cat /etc/resolv.conf
# new DNS server - maintenance 11/02
nameserver 192.0.2.53
dig @10.255.255.254 google.com +short
142.250.188.14
```

Clue	Meaning
`-rw-r--r--` (not `l`)	the managed **link was replaced with a normal file**
`07:14`	when it changed — where to look in the logs
`nameserver 192.0.2.53` + maintenance comment	the bad server, written by hand; search domain gone too
`dig @10.255.255.254 ...` → answer	`@` = ask **this** server, ignoring resolv.conf. Works → DNS is fine, only the **setting** is wrong. Proves the fix before making it

3c. Who changed it? — sudo logs every command

```
journalctl --since "07:00" --no-pager | grep -i "resolv.conf"
07:14:21 sudo[67922]: syedaftab04 : TTY=/dev/pts/10 ; PWD=/home ; USER=root ;
                      COMMAND=/usr/bin/rm /etc/resolv.conf
07:14:21 sudo[67943]: syedaftab04 : TTY=/dev/pts/10 ; PWD=/home ; USER=root ;
                      COMMAND=/usr/bin/tee /etc/resolv.conf
```

Part / field	Meaning
`--since "07:00"`	from just before the file's timestamp
no `-u`	sudo isn't a service — search everything
`grep -i "resolv.conf"`	only lines mentioning the file; `-i` = ignore case
`syedaftab04`	**which person** ran it
`TTY=/dev/pts/10`	which terminal session (pts = SSH / terminal window)
`PWD=/home` / `USER=root`	the folder they were in / run as root
`COMMAND=... rm`, then `... tee`	**exactly what happened**: deleted the link, then wrote a new file

Why sudo instead of logging in as root: every command is logged with a **name** attached. After `su -` to root you'd only see "root did something" — if anything.

Ticket summary: At 07:14 during maintenance, /etc/resolv.conf (a WSL-managed link) was replaced with a hand-written file pointing to 192.0.2.53, which doesn't answer. All DNS lookups time out. Network by IP is fine; the original resolver 10.255.255.254 still works.

STEP 4 Fix — restore the managed link, don't just edit the IP

```
sudo mv /etc/resolv.conf /root/resolv.conf.bad-20261103
sudo ln -s /mnt/wsl/resolv.conf /etc/resolv.conf
ls -l /etc/resolv.conf
lrwxrwxrwx 1 root root 20 Nov  3 07:20 /etc/resolv.conf -> /mnt/wsl/resolv.conf
```

Part	Meaning
`mv FILE /root/...bad-DATE`	**keep the bad file as evidence**, dated, out of the way
`ln -s TARGET LINK`	create a **s**ymbolic link: LINK points to TARGET

Part	Meaning
`/mnt/wsl/resolv.conf`	the target: WSL's own, still-correct file
`/etc/resolv.conf`	the link: where every program looks for DNS settings
`ln -sf`	`-f` = **f**orce: replace an existing file at LINK in one step

Why not just edit the IP? The file would stay **unmanaged**: the next network change (Wi-Fi, VPN) couldn't update it, and DNS would quietly break again. Restoring the link hands control back to its manager.
On a real Ubuntu server with systemd-resolved: `sudo ln -sf /run/systemd/resolve/stub-resolv.conf /etc/resolv.conf`

STEP 5 Verify

```
dig google.com
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 55787
;; ANSWER SECTION:
google.com.             255     IN      A       142.250.188.14
;; Query time: 12 msec
;; SERVER: 10.255.255.254#53(10.255.255.254) (UDP)
ping -c 3 google.com
PING google.com (142.250.188.14) 56(84) bytes of data.  ... 3 received, 0% packet loss
```

Part of dig output	Meaning
`status: NOERROR`	the server answered successfully
`ANSWER SECTION: google.com. 255 IN A 142.250.188.14`	name → IP. `255` = TTL (seconds the answer may be cached), `IN` = internet class, `A` = IPv4 address record
`Query time: 12 msec`	fast — compare with the timeouts before
`SERVER: 10.255.255.254#53`	**the right server** answered

STEP 6 Prevent

1. Change DNS where it's managed, not in resolv.conf:

System	Where DNS is configured	Apply / check
Ubuntu server (netplan + systemd-resolved)	`/etc/netplan/*.yaml` → `nameservers: addresses: [...]`	`sudo netplan apply`, `resolvectl status`
systemd-resolved directly	`/etc/systemd/resolved.conf` (`DNS=`)	`sudo systemctl restart systemd-resolved`, `journalctl -u systemd-resolved`
NetworkManager (desktops, RHEL)	`nmcli con mod CONN ipv4.dns ...`	`nmcli con up CONN`
WSL	`/etc/wsl.conf` `[network]` `generateResolvConf=false`, then manage the file yourself	`wsl --shutdown` to apply
Cloud VMs	usually DHCP from the VPC (AWS Route 53 Resolver, Azure DNS)	change in the VPC/VNet settings, not on the VM

2. Test right after any DNS change — `dig @NEW_SERVER google.com` **before** switching, then `dig google.com` after. The maintenance engineer never tested 192.0.2.53.

3. Check how apps resolve, not just DNS: `getent hosts NAME` follows the same order apps do (`/etc/nsswitch.conf`: usually `/etc/hosts` first, then DNS). `dig` skips /etc/hosts — so a stale /etc/hosts entry shows up in getent but not in dig.

4. Monitor name resolution — a simple check that resolves a known name every minute and alerts on failure catches this before users do.

5. Use sudo, not root shells — that's what made the change traceable in journalctl.

Lab cleanup

```
sudo rm /root/resolv.conf.bad-20261103      # once you no longer need the evidence
```

REF Full command reference

Command	What	Why here
`ls -l /etc/resolv.conf`	is it a link or a file?	found the replaced link
`cat /etc/resolv.conf`	configured nameserver(s)	found 192.0.2.53
`ip -br a`	interfaces + IPs, brief	layer 1
`ip route`	routes / default gateway	layer 2
`ping -c 3 8.8.8.8`	internet by IP	layer 3: network fine
`ping -c 3 NAME`	internet by name	layer 4: DNS broken
`dig NAME` / `+short`	query DNS, full / short	which server, timeout vs NXDOMAIN
`dig @SERVER NAME`	query one specific server	proved the good server works
`journalctl --since "T" | grep -i FILE`	find sudo commands touching a file	who changed it, when, how
`mv FILE /root/FILE.bad-DATE`	keep evidence	before fixing
`ln -s TARGET LINK` / `ln -sf`	create / force a symlink	the fix
`getent hosts NAME`	resolve like apps do (hosts + DNS)	prevention / extra check
`resolvectl status`	systemd-resolved's DNS servers	real-server equivalent
`journalctl -u systemd-resolved`	resolver logs on real servers	real-server equivalent

Interview answer (say this)

"I test bottom-up: ip a for an address, ip route for a default gateway, ping 8.8.8.8 to reach the internet by IP, then ping a name. IP worked but names hung, so it's DNS. dig showed timeouts to 192.0.2.53 — no reply, not NXDOMAIN — so the server itself was wrong or dead. /etc/resolv.conf had been replaced: it was a managed symlink, now a hand-written file with that nameserver. dig @ against the original resolver worked, which proved the fix, and journalctl showed the sudo commands that made the change. I kept the bad file as evidence and restored the symlink rather than editing the IP, so the network manager owns the file again. Prevention: change DNS in netplan or resolved.conf, test with dig before and after, and monitor name resolution."